

VulnHunter: Repository-Level Vulnerability Auditing with Program Analysis and Dynamic Verification

Abstract

Repository-level vulnerability detection requires analyzing cross-function data flows, authorization checks, and project configurations to determine whether suspicious code is reachable and can cause a concrete security impact. DREA advances repository-level vulnerability detection by coordinating planning and exploration agents to gather repository context guided by vulnerability hypotheses. However, whether a candidate path can be triggered in a specific runtime environment, and whether the evidence supporting a verdict can be traced, remain important challenges in vulnerability auditing.

We present VulnHunter, an agent-based framework for auditing Python repositories. Starting from a target function and its repository snapshot, a primary audit agent explores the code context as needed and uses CodeQL, Semgrep, and CodeBadger, a code property graph-based analysis tool, to formulate vulnerability hypotheses for verification. A separate execution agent attempts dynamic reproduction in a task-specific Docker container and records facts and evidence references as append-only events in a blackboard. The system then produces a structured verdict and reports a vulnerability only when a successful attack and its expected security impact have been observed. Comparative results on RepoPairBench show that VulnHunter achieves better overall vulnerability detection performance than DREA. By integrating repository exploration, program analysis, dynamic verification, and evidence recording into a single audit workflow, VulnHunter provides a method for producing verifiable repository-level vulnerability findings.

Keywords: Repository-level vulnerability detection; LLM agents; program analysis; dynamic verification; evidence traceability

1 Introduction

The mechanisms that trigger software vulnerabilities often span multiple functions. Attacker-controlled input may propagate through several calls, while security checks may depend on authorization logic or project configuration elsewhere in the repository. Analyzing only the target function or a fixed code window may therefore fail to reconstruct relevant data flows and calling constraints, making it difficult to determine whether a sensitive operation can be triggered in practice. DREA proposes a framework in which planning and exploration agents collaborate to gather repository context in response to vulnerability hypotheses. It also introduces RepoPairBench for evaluating detection on vulnerable and patched versions of repository code [?].

DREA’s work shows that hypothesis-driven repository exploration can improve vulnerability detection. However, gathering repository context and confirming runtime impact are distinct problems. Static analysis can provide candidate data flows and suspicious operations, but it cannot by itself establish that an attacker can satisfy the conditions required to trigger them. A practical audit must also attempt reproduction in the project environment and record the code locations, actions, and observations that support its verdict. Missing dependencies, an unavailable entry point, or runtime configuration differences may prevent reproduction; such cases should not automatically be treated as evidence that the code is free of vulnerabilities.

Prior work has explored how to combine vulnerability information, program analysis, and dynamic verification. PoCGen starts from known vulnerability reports and combines static analysis for extracting taint paths with dynamic validation of generated proof-of-concept exploits. Its goal is to generate PoCs for disclosed vulnerabilities in npm packages [?]. VulnHunter addresses a different task: starting from a target function and its repository context, it determines whether a vulnerability exists and incorporates dynamic reproduction into the verdict process. Meanwhile, deployment-oriented evaluation has shown that performance on curated benchmarks may not generalize to newer vulnerability samples drawn from a different distribution [?]. Benchmark results should therefore be interpreted in light of the evaluation scope, runtime conditions, and verification coverage.

We present VulnHunter, an agent-based framework for auditing Python repositories. The primary audit agent starts from a target function and its repository snapshot, explores relevant code and call relationships as needed, and uses CodeQL, Semgrep, and CodeBadger, a code property graph-based analysis tool, to formulate vulnerability hypotheses for verification. When dynamic verification is needed, a separate execution agent attempts reproduction in a task-specific Docker container and records observed facts and optional evidence references as append-only events in a blackboard. The primary agent uses the audit findings and recorded facts to produce a structured verdict and report. The system prompt requires a positive verdict only when an attack has succeeded and its expected security impact has been observed.

This work contributes a repository-level auditing workflow that connects repository exploration, program analysis, and container-based dynamic verification; an event-based blackboard for transferring execution observations and evidence references; and a structured report containing the affected location, reproduction steps, verification status, and evidence. We also compare VulnHunter with DREA on RepoPairBench. This draft assumes, as a qualitative result, that VulnHunter achieves better overall detection performance than DREA. It does not attribute this difference to any individual component; that requires experiments with fixed models, budgets, and tool configurations.

2 Problem Definition

2.1 Audit Task

For a vulnerability-fix pair, let R_i^- denote the repository snapshot before the fix and R_i^+ the snapshot after the fix. Let f_i^- and f_i^+ denote the corresponding target functions. The audit system receives a target function and its repository snapshot and makes a decision about vulnerabilities within the scope of that function. The expected labels are $y_i^- = 1$ for the vulnerable version and $y_i^+ = 0$ for the patched version.

The system produces a binary verdict $\hat{y}_i \in \{0, 1\}$. For a positive verdict, it must also provide a vulnerability report containing the affected location, preconditions, reproduction steps, expected and observed security impact, and supporting evidence references. Evaluation is restricted to the target function specified by the dataset, so that unrelated vulnerabilities elsewhere in the repository do not count as correct findings for the sample.

2.2 Clues, Hypotheses, and Observations

We distinguish three concepts in the audit process. A **clue** is a suspicious operation, data flow, or call relationship identified through code inspection or a program analysis tool. A **hypothesis** is an explanation of a vulnerability based on input control, propagation paths, protective conditions, and potential impact. An **observation** is behavior actually recorded by the execution agent in a runtime environment. Clues can motivate hypotheses, and hypotheses can guide verification, but neither substitutes for runtime observations.

Based on this distinction, we investigate the following questions:

- **RQ1:** Does VulnHunter achieve better overall vulnerability detection performance than DREA on RepoPairBench?
- **RQ2:** Can structured fact events and evidence references connect the execution agent’s observations to the final report?
- **RQ3:** How does requiring observed attack success for a positive verdict affect detection coverage and the handling of cases where reproduction cannot be completed?

3 VulnHunter System Design

3.1 System Workflow

VulnHunter consists of a primary audit agent, an execution agent, program analysis tools, and a fact blackboard. The primary agent interprets the target function, explores repository context, forms vulnerability hypotheses, and prepares verification plans. When runtime verification is needed, it delegates a task related to the current hypothesis to the execution agent. The execution agent operates in the designated container and writes facts and optional evidence references to the blackboard. The primary agent then reads those records, summarizes the audit findings, and produces a structured result.

For target-function auditing, the system takes a repository snapshot, file path, and target function as input. The primary agent may inspect the target file and, as needed for the current hypothesis, its callers, callees, configuration, and tests. Evaluation scores only the specified target function; findings elsewhere in the repository do not change the sample’s label.

3.2 Repository Exploration and Program Analysis

The primary audit agent determines which files and call contexts to inspect based on its current hypothesis. It can query CodeQL, Semgrep, and CodeBadger as needed. CodeQL and Semgrep provide different forms of static analysis evidence, while CodeBadger provides query capabilities based on Joern’s code property graph. Code property graphs provide a unified representation of program structures such as syntax, control flow, and data flow, and can express relationships across these structures [?].

Tool outputs support or refine hypotheses; they are not directly converted into final verdicts. The system does not require every tool to be invoked for every task, nor does it use the number of alerts as a voting mechanism. Final decisions combine repository context with dynamic verification results.

3.3 Dynamic Verification Agent

The execution agent receives a vulnerability hypothesis and verification plan from the primary agent. It uses Docker MCP tools to perform operations directly related to the hypothesis inside a task-specific container. These operations may include checking dependencies, constructing inputs, running target code, and observing potential security impacts. The execution agent does not inherit the primary agent’s host filesystem tools; its container access is determined by the Docker tools provided to it.

The system prompt requires the execution agent to distinguish confirmed, unconfirmed, and inconclusive outcomes, and to append facts to the blackboard when its task ends. An outcome can be marked as confirmed only if the agent carries out the attack and observes the expected impact. The container provides an execution environment and resource attribution for the task; this paper does not characterize it as a completely closed security sandbox.

3.4 Event-Based Fact Blackboard

The blackboard records facts and optional evidence references submitted by the execution agent as events. Each event has an identifier. Events are deduplicated by identifier during merging, and the system rejects conflicting contents associated with the same identifier rather than silently overwriting them. The primary audit agent reconstructs readable blackboard context from the event collection, preventing stale text snapshots returned by concurrent tasks or retries from overwriting previously appended facts.

The event mechanism ensures consistent record merging; it does not guarantee that the agent’s statements are true. Blackboard events may omit evidence references, while a positive vulnerability report requires at least one evidence item. References in reports must therefore still be checked against the original code, command output, and tool results.

3.5 Structured Verdicts and Resource Cleanup

An audit result contains a binary verdict and an optional vulnerability report. For positive verdicts, the report must include a summary, affected location, preconditions, reproduction steps, expected impact, observed impact, verification status, and evidence. The structured model validates the relationship between the verdict and the report and checks that the required fields are present for a positive finding.

The current verdict policy requires a recorded successful attack and expected security impact for a positive result. When reproduction is unsuccessful, the prompt instructs the model to return a negative verdict. This policy clarifies the evidence threshold for positive findings, but it may classify environmental failures or incomplete verification as negative; this limitation must be accounted for when evaluating and interpreting results.

The system assigns separate checkout directories and containers to evaluation samples. When a task completes, fails, or is cancelled, the cleanup workflow releases associated resources and removes the task container. Evaluation paths that enable checkout cleanup also remove the sample checkout. Resource cleanup supports task management; it does not establish detection accuracy or complete security isolation.

4 Evaluation Design

4.1 Dataset and Execution Setup

We use RepoPairBench, introduced by DREA. The benchmark consists of public Python vulnerability-fix pairs and provides the vulnerable and patched repository snapshots along with their corresponding target functions. It is designed for comparing verdicts on both sides of a fix. The evaluation service retrieves code using repository addresses and commit references, then provides the target path and function to the audit task.

A formal comparison should fix VulnHunter’s code version, primary model and inference parameters, static tool versions, container image, execution budget, and concurrency settings. It should also preserve each sample’s final status and tool-call trace. Because public fix commits may contain label clues that tools could read indirectly, experiments should check whether agents or tools accessed the patch, commit message, or other information that reveals the answer.

4.2 Baselines and Scope of Comparison

DREA is the primary comparison system. Its paper reports detection metrics and paired outcomes on RepoPairBench [?]. Comparisons should align the target functions, vulnerable versions, and patched versions as closely as possible. If the primary model, tool budget, or verdict policy is not aligned, the results can describe system-level performance differences, but cannot establish that a particular agent architecture or tool caused them.

PoCGen is not a direct detection baseline for this paper. It takes a known vulnerability report as input and aims to generate and validate PoCs for npm package vulnerabilities, whereas VulnHunter evaluates whether a target function is vulnerable. The tasks differ in their inputs and outputs, although PoCGen demonstrates how static information and runtime feedback can support vulnerability reproduction [?].

4.3 Metrics

We use vulnerable-side recall, patched-side false-positive rate, precision, F_1 , and accuracy to evaluate individual verdicts. Let TP and FN denote correct positive verdicts and false negatives for vulnerable versions; let FP and TN denote false positives and correct negative verdicts for patched versions. Then:

$$\begin{aligned}\text{Recall} &= \frac{TP}{TP + FN}, & \text{FPR} &= \frac{FP}{FP + TN}, \\ \text{Precision} &= \frac{TP}{TP + FP}, & F_1 &= \frac{2TP}{2TP + FP + FN}.\end{aligned}$$

To measure consistency across both sides of a fix, we use Pair-Correctness. A pair counts as correct only when the vulnerable version is classified as positive and the patched version as negative. The evaluation should also report valid verdict coverage, unparseable verdicts, execution failures, and cancellations. Only pairs with valid verdicts on both sides are included in the Pair-Correctness denominator.

4.4 Qualitative Comparison Results

This draft reports the following qualitative comparison: VulnHunter achieves better overall vulnerability detection performance than DREA on RepoPairBench. Because this draft does not include metric values, it does not claim a specific advantage in recall, false-positive rate, or Pair-Correctness, nor does it claim that any individual tool or dynamic verification step alone produces the improvement. Reproduction experiments with fixed configurations and internal ablations are needed to analyze the contributions of individual components.

5 Discussion and Threats to Validity

5.1 Dynamic Confirmation and Inconclusive Cases

Requiring an observed attack and its expected impact helps prevent positive verdicts based solely on suspicious static paths and ties vulnerability reports to observed behavior. However, when execution fails,

dependencies are missing, or a reproduction entry point cannot be identified, the absence of an observed impact does not necessarily mean that the vulnerability is absent. Because the current structured result is binary, these cases may be recorded as negative and affect vulnerable-side recall.

A complete evaluation should preserve sample execution states and report how many cases could not be verified, along with the reasons. Binary metrics should distinguish samples confirmed to be non-vulnerable from those that could not be confirmed because of environmental or verification constraints. Future versions could expose an explicit “inconclusive” result, but this would require corresponding changes to metric definitions and to comparisons with DREA.

5.2 Evidence Recording and Evidence Validity

The append-only blackboard reduces the risk of losing recorded facts during concurrent execution or retries, and allows the primary agent to read the execution agent’s observations. It addresses event recording and merging; it does not validate whether event statements reflect actual execution. Evidence references may also be insufficient or point to irrelevant outputs. The traceability of positive reports must therefore be checked against original agent traces, command outputs, and code locations.

5.3 Fairness of Comparisons

Comparisons across systems may be affected by the backbone model, prompt, call budget, tool access, runtime environment, and positive-verdict policy. Even when DREA and VulnHunter use the same benchmark, differences in published results alone do not establish that a particular architectural component caused a performance change. Future experiments should prioritize reproducing DREA with aligned models and budgets and report the actual completion counts and failure-handling rules.

5.4 External Validity and Generalization

RepoPairBench focuses on public Python vulnerability-fix pairs and does not cover other languages, undisclosed vulnerabilities, or proprietary code. *From Lab to Reality* reports that benchmark performance in controlled settings may not reflect performance on newer vulnerability samples with different distributions [?]. VulnHunter’s results on RepoPairBench therefore cannot, by themselves, establish its effectiveness in real-world deployment. Further evaluation is needed on newer vulnerabilities, different projects, and varied runtime environments.

6 Related Work

6.1 Agent-Based Repository-Level Vulnerability Detection

DREA assigns vulnerability reasoning and repository exploration to planning and exploration agents. This allows the system to gather context on demand around current hypotheses and introduces RepoPairBench for paired evaluation of vulnerable and patched code [?]. VulnHunter likewise targets repository-level auditing, with an emphasis on passing program analysis clues to an execution agent for dynamic verification and connecting its observations to the final verdict through fact events.

ReAct organizes language model reasoning and environmental actions in an alternating process, providing a general interaction pattern for tool-using agents [?]. SWE-agent explores computer interfaces through which agents can operate on code repositories [?]. These works provide context for agent interaction with code environments; VulnHunter focuses on connecting vulnerability hypotheses, runtime verification, and audit evidence.

6.2 PoC Generation and Dynamic Verification

PoCGen extracts vulnerability types, target functions, and data-flow information from existing vulnerability reports, then combines a language model, static analysis, and runtime checks to generate and validate PoCs for npm package vulnerabilities [?]. It assumes that a vulnerability has already been reported and focuses on constructing an executable exploit. VulnHunter treats vulnerability assessment as the task itself and uses dynamic reproduction as a condition for confirming a positive verdict. Both works use static and runtime information, but differ in their inputs, evaluation goals, and outputs.

6.3 Deployment-Oriented Vulnerability Detection Evaluation

From Lab to Reality compares deep learning and large language models on established benchmarks and temporally newer vulnerability samples, emphasizing a potential generalization gap between benchmark data and deployment scenarios [?]. That work examines models across datasets and distributions, while this paper uses RepoPairBench to evaluate repository-level agents on vulnerable and patched target functions. These evaluations are complementary, but results on RepoPairBench cannot replace evaluation on novel vulnerabilities or in real deployment environments.

6.4 Program Analysis Representations

Code property graphs represent multiple program structures in a unified graph and can support cross-structural program analysis and vulnerability discovery [?]. VulnHunter uses CodeQL, Semgrep, and CodeBadger to obtain program analysis clues, but tool alerts do not directly determine the final verdict. The system forms its conclusions by combining those clues with code context and dynamic observations.

7 Conclusion

This paper presents VulnHunter, an agent-based framework for auditing Python repositories. It integrates on-demand repository exploration, multi-source program analysis, container-based dynamic reproduction, append-only fact recording, and structured verdicts into a single workflow. Positive verdicts require an observed attack and its expected impact. This draft qualitatively assumes that VulnHunter achieves better overall performance than DREA on RepoPairBench. Future work should add reproducible experiments with fixed configurations, per-metric results, ablation analyses, and evidence audits. It should also handle cases where reproduction cannot be completed separately to establish the scope of this claim.